

AICE3001/AICE6002: Mathematics of Machine Learning

Subsidiary Notes — Lecture 5: Inner Product Spaces

Adam Prugel-Bennett

September 29, 2026

1 Keywords

- Inner-product, operators

2 Main Points

- For some vector spaces we can define an *inner product* between pairs of vectors
- Inner products are scalars associated with two elements in a vector space
- They are generally denoted by $\langle \mathbf{x}, \mathbf{y} \rangle$
- To be an inner-product requires satisfying 5 axioms

1. $\langle \mathbf{x}, \mathbf{x} \rangle \geq 0$ for all $\mathbf{x} \in \mathcal{V}$
2. $\langle \mathbf{x}, \mathbf{x} \rangle = 0$ if and only if $\mathbf{x} = \mathbf{0}$
3. $\langle \alpha \mathbf{x}, \mathbf{y} \rangle = \alpha \langle \mathbf{x}, \mathbf{y} \rangle$
4. $\langle \mathbf{x}, \mathbf{y} + \mathbf{z} \rangle = \langle \mathbf{x}, \mathbf{y} \rangle + \langle \mathbf{x}, \mathbf{z} \rangle$
5. $\langle \mathbf{x}, \mathbf{y} \rangle = \langle \mathbf{y}, \mathbf{x} \rangle$

- The inner-product induces a norm: $\|\mathbf{x}\| = \sqrt{\langle \mathbf{x}, \mathbf{x} \rangle}$
- For normal vectors (i.e. $\mathbf{x} \in \mathbb{R}^n$) the standard inner product is the dot-product

$$\langle \mathbf{x}, \mathbf{y} \rangle = \mathbf{x}^\top \mathbf{y} = \sum_{i=1}^n x_i y_i$$

- We can define an inner product between functions

$$\langle f, g \rangle = \int_{x \in \mathcal{I}} f(x) g(x) \, dx$$

- For matrices we can define the Frobenius inner-product

$$\langle \mathbf{A}, \mathbf{B} \rangle_F = \text{Tr } \mathbf{A}^\top \mathbf{B} = \sum_{i,j} A_{ij} B_{ij}$$

which defines the Frobenius norm

$$\|\mathbf{A}\|_F = \sqrt{\langle \mathbf{A}, \mathbf{A} \rangle_F} = \sqrt{\sum_{i,j} A_{ij}^2}$$

The Frobenius norm is compatible with the Euclidean norm ($\|\mathbf{A}\mathbf{x}\| \leq \|\mathbf{A}\|_F \|\mathbf{x}\|$), but it is not *induced* by any vector norm, unlike the spectral norm.

- A really important inequality is the *Cauchy-Schwarz* inequality

$$\langle \mathbf{x}, \mathbf{y} \rangle^2 \leq \langle \mathbf{x}, \mathbf{x} \rangle \langle \mathbf{y}, \mathbf{y} \rangle = \|\mathbf{x}\|^2 \|\mathbf{y}\|^2$$

- Inner products allow us to define the notion of similarity

$$\begin{aligned} \langle \mathbf{x}, \mathbf{y} \rangle &= \|\mathbf{x}\| \|\mathbf{y}\| \cos(\theta) \\ \langle f(x), g(x) \rangle &= \|f(x)\| \|g(x)\| \cos(\theta) \end{aligned}$$

- $\cos(\theta)$ can be seen as a measure of the correlation between vectors (or functions)
- Because of Cauchy-Schwarz $\langle \mathbf{x}, \mathbf{y} \rangle / (\|\mathbf{x}\| \|\mathbf{y}\|)$ lies between -1 and 1 (so that we can represent this quantity by the cosine of an angle)

2.1 Coordinates or Basis Vectors

- Any set of vectors that span the entire vector space can be considered a set of basis vectors or coordinates
- If our bases are linearly independent then we have a set of non-degenerate basis functions where each vector is unique
- The most convenient set of basis vectors are those where the bases are normalised and orthogonal $\langle \mathbf{b}_i, \mathbf{b}_j \rangle = \delta_{ij}$

• Basis Functions

- For a function space we can consider a set of basis functions
- A familiar set of functions defined on the interval $[0, 2\pi]$ are the Fourier functions

$$\{1, \cos(\theta), \sin(\theta), \cos(2\theta), \sin(2\theta), \dots\}$$

- This set is orthogonal: $\langle b_i(\theta), b_j(\theta) \rangle = 0$ whenever $i \neq j$. It is not normalised — for example $\langle 1, 1 \rangle = 2\pi$ while $\langle \cos \theta, \cos \theta \rangle = \pi$ — so divide each by its own norm to get an *orthonormal* set
- There are many orthogonal polynomials that have similar properties
- Given an *orthonormal* set of functions $\{b_i(x)\}$ we can decompose a function $f(x)$ as an (infinite) vector $\mathbf{f}$ with components $f_i = \langle f(x), b_i(x) \rangle$. If the set is orthogonal but not normalised the components are $f_i = \langle f, b_i \rangle / \langle b_i, b_i \rangle$
- This allows us to represent any function as a point in an infinite-dimensional space

2.2 Projection Operators

- Given a vector $\mathbf{a}$ we can ask how much of it lies along another vector $\mathbf{b}$

$$\text{proj}_{\mathbf{b}}(\mathbf{a}) = \frac{\langle \mathbf{a}, \mathbf{b} \rangle}{\langle \mathbf{b}, \mathbf{b} \rangle} \mathbf{b}$$

- Read this as two steps: the scalar $\langle \mathbf{a}, \mathbf{b} \rangle / \langle \mathbf{b}, \mathbf{b} \rangle$ says how many copies of $\mathbf{b}$ are needed, and multiplying by $\mathbf{b}$ puts the answer back in the direction of $\mathbf{b}$
- If $\mathbf{b}$ is normalised ($\|\mathbf{b}\| = 1$) this is just $\langle \mathbf{a}, \mathbf{b} \rangle \mathbf{b}$
- Using $\langle \mathbf{a}, \mathbf{b} \rangle = \|\mathbf{a}\| \|\mathbf{b}\| \cos(\theta)$ the length of the projection is $\|\mathbf{a}\| \cos(\theta)$, which is the picture you would draw

- The defining property of a projection is that doing it twice changes nothing — it is *idempotent*

$$\text{proj}_{\mathbf{b}}(\text{proj}_{\mathbf{b}}(\mathbf{a})) = \text{proj}_{\mathbf{b}}(\mathbf{a})$$

which follows because $\text{proj}_{\mathbf{b}}(\mathbf{a})$ already points along $\mathbf{b}$, so projecting it again leaves it alone

- The **remainder** is what matters for what follows: $\mathbf{a} - \text{proj}_{\mathbf{b}}(\mathbf{a})$ is orthogonal to $\mathbf{b}$, since

$$\left\langle \mathbf{a} - \frac{\langle \mathbf{a}, \mathbf{b} \rangle}{\langle \mathbf{b}, \mathbf{b} \rangle} \mathbf{b}, \mathbf{b} \right\rangle = \langle \mathbf{a}, \mathbf{b} \rangle - \frac{\langle \mathbf{a}, \mathbf{b} \rangle}{\langle \mathbf{b}, \mathbf{b} \rangle} \langle \mathbf{b}, \mathbf{b} \rangle = 0$$

- For vectors in $\mathbb{R}^n$ the projection is a matrix, $\mathbf{P} = \mathbf{b} \mathbf{b}^T / (\mathbf{b}^T \mathbf{b})$, and idempotency reads $\mathbf{P}^2 = \mathbf{P}$
- To project onto a whole subspace spanned by an orthonormal set $\{\mathbf{e}_1, \dots, \mathbf{e}_k\}$ just add the projections, $\mathbf{P} = \sum_{i=1}^k \mathbf{e}_i \mathbf{e}_i^T$. This is the operator that appears again in PCA

2.3 Gram-Schmidt Orthogonalisation

- Projections let us turn any set of linearly independent vectors $(\mathbf{a}_1, \dots, \mathbf{a}_n)$ into an orthonormal basis
- Take each vector in turn and subtract off everything it shares with the directions already found; normalise what is left

$$\begin{aligned} \mathbf{u}_1 &= \mathbf{a}_1, & \mathbf{e}_1 &= \frac{\mathbf{u}_1}{\|\mathbf{u}_1\|} \\ \mathbf{u}_2 &= \mathbf{a}_2 - \text{proj}_{\mathbf{u}_1}(\mathbf{a}_2), & \mathbf{e}_2 &= \frac{\mathbf{u}_2}{\|\mathbf{u}_2\|} \\ \mathbf{u}_3 &= \mathbf{a}_3 - \text{proj}_{\mathbf{u}_1}(\mathbf{a}_3) - \text{proj}_{\mathbf{u}_2}(\mathbf{a}_3), & \mathbf{e}_3 &= \frac{\mathbf{u}_3}{\|\mathbf{u}_3\|} \end{aligned}$$

and in general $\mathbf{u}_k = \mathbf{a}_k - \sum_{i < k} \text{proj}_{\mathbf{u}_i}(\mathbf{a}_k)$

- **The projections are onto the $\mathbf{u}_i$, not the $\mathbf{a}_i$.** This is the easy mistake. Subtracting $\text{proj}_{\mathbf{a}_2}(\mathbf{a}_3)$ instead of $\text{proj}_{\mathbf{u}_2}(\mathbf{a}_3)$ does *not* leave something orthogonal to both, because $\mathbf{a}_2$ is not itself orthogonal to $\mathbf{a}_1$. Only $\mathbf{u}_2 = \mathbf{a}_2$ when $k = 2$, which is why the second step looks the same either way
- Each $\mathbf{u}_k$ is orthogonal to everything before it by construction: it is a remainder, and we showed above that remainders are orthogonal to what was subtracted
- If the $\mathbf{a}_i$ are *not* linearly independent some $\mathbf{u}_k$ comes out as $\mathbf{0}$, and the process tells you the set was degenerate
- The same algorithm works in any inner-product space. Applied to $1, x, x^2, \dots$ with $\langle f, g \rangle = \int f g \, dx$ it generates the orthogonal polynomials
- In $\mathbb{R}^n$ it is numerically poor: rounding errors accumulate and the later vectors drift away from orthogonality. Numerical libraries compute the same factorisation with Householder reflections instead. Gram-Schmidt is how you *understand* the result, not how you compute it

2.4 Operators

- Operators transform elements of a vector space
- Consider the transformation or operator $\mathcal{T} : \mathcal{V} \rightarrow \mathcal{V}'$

- This says that $\mathcal{T}$ maps some object $x \in \mathcal{V}$ to an object $y = \mathcal{T}[x]$ in a new vector space $\mathcal{V}'$

- **Linear Operators**

- Linear operators satisfy the two conditions
 1. $\mathcal{T}[a x] = a \mathcal{T}[x]$
 2. $\mathcal{T}[x + y] = \mathcal{T}[x] + \mathcal{T}[y]$
- Linear operators are really important because we can understand them
- For normal vectors the most general linear operation is

$$\mathcal{T}[x] = \mathbf{M} x$$

where $\mathbf{M}$ is a matrix

- For functions the most general linear operator is a kernel function

$$g(x) = \mathcal{T}[f(x)] = \int K(x, y) f(y) dy$$

– Kernels appear in SVMs, SVRs, kernel-PCA, Gaussian Processes

- Often we are interested in operators that map vectors in a vector space to new vectors in the same vector space
 - $\mathcal{T} : \mathcal{V} \rightarrow \mathcal{V}$
 - The most general linear mapping for normal vectors that does this is a square matrix

2.5 Matrices and Mappings

- Matrices, $\mathbf{M}$ are linear maps from one point x to another $y = \mathbf{M}x$
- The product of a matrix $\mathbf{C} = \mathbf{A}\mathbf{B}$ corresponds to applying the mapping $\mathbf{B}$ followed by $\mathbf{A}$
- For most matrices $\mathbf{A}\mathbf{B} \neq \mathbf{B}\mathbf{A}$ (we say matrix multiplication does not commute)
- There are pairs of matrices that do commute, but they need to share a special structure for this to happen
- Matrix multiplication is associative. That is, $(\mathbf{A}\mathbf{B})\mathbf{C} = \mathbf{A}(\mathbf{B}\mathbf{C})$